전이학습(Feature Extraction) 이미지 분류 파이프라인 구현 👩‍💻

신발,샌들,부츠 이미지 데이터셋 (Shoe vs Sandal vs Boot Image Dataset)

본 프로젝트는 전이 학습의 특징 추출(Feature Extraction) 방식을 활용하여, 신발·샌들·부츠 이미지 데이터셋(Shoe vs. Sandal vs. Boot)을 기반으로 3개 클래스를 정확하게 구별하는 다중 분류 모델의 학습 및 평가 파이프라인을 구축하는 것을 목표로 합니다.

- [데이터셋 링크] https://www.kaggle.com/datasets/hasibalmuzdadid/shoe-vs-sandal-vs-boot-dataset-15k-images

데이터 준비

```
base_path = r"C:\kdt_0900_hhj\ml\resource\dataset"
folder_name = r"SSBDataset"
file_path = os.path.join(base_path, folder_name)
file_path
```

```
from kaggle.api.kaggle_api_extended import KaggleApi

api = KaggleApi()
api.authenticate()

os.makedirs(base_path, exist_ok=True)
api.dataset_download_files(r"hasibalmuzdadid/shoe-vs-sandal-vs-boot-dataset-15k-images", path=base_path, unzip=True)

print("다운로드 완료")
```

데이터 나누기

```
base_dataset = ImageFolder(root=file_path, transform=None)
```

```
total_size = len(base_dataset)
train_size = int(total_size * 0.8)
test_size = total_size - train_size
```

```
print(train_size, test_size)
```

```
train_subset, test_subset = random_split(base_dataset, [tra
in_size, test_size], generator=torch.Generator().manual_see
d(42))
```

```
print(len(train_subset), len(test_subset))
```

```
12000 3000
```

```
# MobileNetV2가 학습할 때 사용하는 이미지 정규화 기준값. 이미지 픽
셀값을 정규화해서 MobileNet이 학습하기 좋은 형태로 맞춰준다
mean = [0.485, 0.456, 0.406]
std = [0.229, 0.224, 0.225]
```

```
train_transform = transforms.Compose([
    transforms.Resize((224, 224)),
    transforms.RandomHorizontalFlip(),
    transforms.RandomRotation(15),
    transforms.ToTensor(),
    transforms.Normalize(mean=mean, std=std)
])
test_transform = transforms.Compose([
    transforms.Resize(256),
    transforms.CenterCrop(224),
    transforms.ToTensor(),
```

```
    transforms.Normalize(mean=mean, std=std)
])
```

```
class DatasetWrapper(Dataset):
    def __init__(self, dataset, transform=None):
        self.dataset = dataset
        self.transform = transform

    def __len__(self):
        return len(self.dataset)

    def __getitem__(self, idx):
        x, y = self.dataset[idx]
        if self.transform:
            x = self.transform(x)
        return x, y
```

```
train_dataset = DatasetWrapper(train_subset, transform=trai
n_transform)
test_dataset = DatasetWrapper(test_subset, transform=test_t
ransform)
```

```
print(len(train_dataset), len(test_dataset))
```

```
12000 3000
```

```
class_dict = dict(enumerate(base_dataset.classes))
print(f"클래스 수: {class_dict}")
print(f"라벨 매핑: {base_dataset.class_to_idx}")
print(f"학습 데이터 샘플 수: {len(train_dataset)}")
print(f"검증 데이터 샘플 수: {len(test_dataset)}")
```

```
클래스 수: {0: 'Boot', 1: 'Sandal', 2: 'Shoe'}
라벨 매핑: {'Boot': 0, 'Sandal': 1, 'Shoe': 2}
학습 데이터 샘플 수: 12000
검증 데이터 샘플 수: 3000
```

```
train_dataloader = DataLoader(train_dataset, batch_size=32, shuffle=True)
test_dataloader = DataLoader(test_dataset, batch_size=32, shuffle=False)
```

```
imgs, labels = next(iter(train_dataloader))
```

```
print(imgs.size())
```

```
torch.Size([32, 3, 224, 224])
```

배치크기 32, RGB 색상 채널 3개, 이미지크기 가로세로 224x224

확인해보기

```
mean = torch.tensor([0.485, 0.456, 0.406]).view(3, 1, 1)
std = torch.tensor([0.229, 0.224, 0.225]).view(3, 1, 1)

fig, axes = plt.subplots(3, 5, figsize=(8, 5))

for ax, img, label in zip(axes.flatten(), imgs, labels):
    img = img.cpu() * std + mean # 정규화 이전의 픽셀값으로 복원
    img = img.permute(1, 2, 0)
    ax.imshow(img)
    ax.set_title(class_dict[label.item()])
    ax.axis("off")

plt.show()
```



장치 선택

```
if torch.cuda.is_available():
    device = torch.device("cuda")
elif torch.mps.is_available():
    device = torch.device("mps")
elif torch.xpu.is_available():
    device = torch.device("xpu")
else:
    device = torch.device("cpu")

print(f"현재 선택된 장치: {device}")
```

```
현재 선택된 장치: cuda
```

모델 준비

```
model = models.mobilenet_v2(weights=models.MobileNet_V2_Weights.DEFAULT)
```

모델 선정 이유

데이터셋 특징

- 신발 이미지를 종류별로 분류하는 다중 클래스 이미지 분류 문제이다.
- 클래스는 3개, 이미지는 약 15,000장, 클래스마다 5,000장이 있다.
- 클래스별 데이터 수가 같아 데이터 불균형 문제는 적을 것으로 예상한다.
- 데이터셋 이미지의 촬영 각도와 배경이 일정하므로 배경이나 각도보다는 신발의 형태와 디자인 차이를 학습하는 것이 중요하다고 판단하였다.

선정 이유

- MobileNetV2는 ResNet과 같은 모델에 비해 비교적 가볍고 계산량이 적어, 모델의 크기와 실행 효율을 고려하여 선택하였다.
- 또한 사전 학습된 가중치를 고정하고 마지막 분류층만 학습하면 학습해야 하는 파라미터 수를 줄일 수 있으므로 구글 코랩(Colab T4) 환경이나 개인 PC의 로컬 GPU/MPS 환경에서 학습할 때, 모델 용량이 가볍고 연산이 빨라 주어진 환경에서 효율적으로 학습할 수 있을 것으로 판단하였다.

다만, 데이터셋 이미지가 동일한 각도와 단조로운 배경 조건을 가지고 있어 학습 데이터에만 과적합(Overfitting)될 위험이 있으므로 데이터 전처리 단계에서 미세한 회전이나 크기 조절 등의 데이터 증강(Data Augmentation)을 적용해 대응하고자 한다.

모델 프리징

```
for param in model.parameters():
    param.requires_grad = False
```

모델 수정

분류할 클래스가 3개라 out_features은 수정해야 한다

```
model.classifier = nn.Sequential(
    nn.Dropout(p=0.2, inplace=False),
    nn.Linear(model.classifier[1].in_features, 256),
    nn.BatchNorm1d(256),
    nn.ReLU(),
    nn.Dropout(0.4),
```

```
        nn.Linear(256, 3)
    )
```

```
print(model)
```

```
    )
    (classifier): Sequential(
      (0): Dropout(p=0.2, inplace=False)
      (1): Linear(in_features=1280, out_features=256, bias=True)
      (2): BatchNorm1d(256, eps=1e-05, momentum=0.1, affine=True, track_running_stats=True)
      (3): ReLU()
      (4): Dropout(p=0.4, inplace=False)
      (5): Linear(in_features=256, out_features=3, bias=True)
    )
)
```

확인

```
# 손실 함수
criterion = nn.CrossEntropyLoss()
# 최적화 도구
optimizer = optim.Adam(model.classifier.parameters(), lr=0.001)
```

```
model = model.to(device)
```

학습

```
def train_one_epoch_shoes(model, dataloader, criterion, optimizer, device="cpu"):
    model.train() # 학습모드
    sum_losses = 0.0
    sum_accs = 0.0

    for i, (X_batch, y_batch) in enumerate(dataloader, 1):
        X_batch, y_batch = X_batch.to(device), y_batch.to(device)
        batch_loss = 0.0

        # 학습
```

```
        optimizer.zero_grad()
        y_pred = model(X_batch)
        loss = criterion(y_pred, y_batch)
        loss.backward()
        optimizer.step()

        batch_loss += loss.item()
        sum_losses += batch_loss

        # 다중 클래스의 정확도 계산
        y_pred_index = y_pred.argmax(dim=1)
        acc = (y_batch == y_pred_index).float().mean().item
() * 100
        sum_accs += acc

        if i % 20 == 0:
            print(f"[Batch {i}/{len(dataloader)}] Batch Los
s: {batch_loss:.4f}, Batch Accuracy: {acc:.2f}%")

    avg_loss = sum_losses / len(dataloader)
    avg_acc = sum_accs / len(dataloader)

    return avg_loss, avg_acc
```

```
@torch.no_grad() # 기울기를 계산x
def evaluate_shoes(model, dataloader, criterion, device="cp
u"):
    model.eval()
    sum_losses = 0.0
    sum_accs = 0.0

    for X_batch, y_batch in dataloader:
        X_batch, y_batch = X_batch.to(device), y_batch.to(d
evice)

        y_pred = model(X_batch)
        loss = criterion(y_pred, y_batch)
        sum_losses += loss.item()
```

```
        # 다중 클래스 정확도 계산
        y_pred_index = y_pred.argmax(dim=1)
        acc = (y_batch == y_pred_index).float().mean().item
() * 100
        sum_accs += acc

    avg_loss = sum_losses / len(dataloader)
    avg_acc = sum_accs / len(dataloader)

    return avg_loss, avg_acc
```

```
def train_model_shoes(model, model_name, train_loader, val_
loader, criterion, optimizer, epochs, device, save_base_di
r):
    save_dir = os.path.join(save_base_dir, model_name)
    os.makedirs(save_dir, exist_ok=True)

    history = {"train_loss": [], "val_loss": [], "train_ac
c": [], "val_acc": []}
    best_val_acc = 0.0

    # 학습
    print(f"========================= {model_name} 학습 시작
============================")
    for epoch in range(epochs):
        train_loss, train_acc = train_one_epoch_shoes(mode
l, train_loader, criterion, optimizer, device)
        val_loss, val_acc = evaluate_shoes(model, val_loade
r, criterion, device)

        print(f"[Epoch: {epoch + 1}] | Train Loss: {train_l
oss:.4f} | Train Accuracy: {train_acc:.2f}%")
        print(f"[Epoch: {epoch + 1}] | Validation Loss: {va
l_loss:.4f} | Validation Accuracy: {val_acc:.2f}%")

        history["train_loss"].append(train_loss)
        history["val_loss"].append(val_loss)
```

```
        history["train_acc"].append(train_acc)
        history["val_acc"].append(val_acc)

        if val_acc > best_val_acc:
            best_val_acc = val_acc
            torch.save(model.state_dict(), os.path.join(sav
e_dir, "best_model.pth"))

    torch.save(model.state_dict(), os.path.join(save_dir,
f"last_model_{epochs}.pth"))
    print(f"======================== {model_name} 학습 종료
============================")
    return history
```

```
history = train_model_shoes(
    model=model,
    model_name="Mobilenet_shoes",
    train_loader=train_dataloader,
    val_loader=test_dataloader,
    criterion=criterion,
    optimizer=optimizer,
    epochs=20,
    device=device,
    save_base_dir=os.path.join(base_path, "weights")
)
```

```
[Batch 160/375] Batch Loss: 0.3130, Batch Accuracy: 87.50%
[Batch 180/375] Batch Loss: 0.1558, Batch Accuracy: 93.75%
[Batch 200/375] Batch Loss: 0.0478, Batch Accuracy: 96.88%
[Batch 220/375] Batch Loss: 0.1068, Batch Accuracy: 90.62%
[Batch 240/375] Batch Loss: 0.0440, Batch Accuracy: 96.88%
[Batch 260/375] Batch Loss: 0.1965, Batch Accuracy: 90.62%
[Batch 280/375] Batch Loss: 0.0953, Batch Accuracy: 96.88%
[Batch 300/375] Batch Loss: 0.1296, Batch Accuracy: 93.75%
[Batch 320/375] Batch Loss: 0.2360, Batch Accuracy: 90.62%
[Batch 340/375] Batch Loss: 0.3528, Batch Accuracy: 81.25%
[Batch 360/375] Batch Loss: 0.2034, Batch Accuracy: 90.62%
[Epoch: 20] | Train Loss: 0.1610 | Train Accuracy: 93.73%
[Epoch: 20] | Validation Loss: 0.2221 | Validation Accuracy: 92.11%
======================== Mobilenet_shoes 학습 종료 ============================
```

학습 정확도는 93.73%, 검증 정확도는 92.11%로 두 지표의 차이가 크지 않았다.

평가

```
best_model_path = os.path.join(base_path, r"weights", r"Mob
ilenet_shoes", r"best_model.pth")

model.load_state_dict(
    torch.load(best_model_path, map_location=device, weight
s_only=True)
)

model = model.to(device)
model.eval()

print("최적 모델 로드 완료")
```

Accuracy 측정

```
test_loss, test_acc = evaluate_shoes(
    model=model,
    dataloader=test_dataloader,
    criterion=criterion,
    device=device
)

print(f"Test Loss: {test_loss:.4f}")
print(f"Test Accuracy: {test_acc:.2f}%")
```

```
Test Loss: 0.2079
Test Accuracy: 92.40%
```

예측 시각화하기

```
imgs, labels = next(iter(test_dataloader))
```

```
model.eval()
imgs_device = imgs.to(device)

with torch.no_grad():
    preds = model(imgs_device)
    pred_labels = preds.argmax(dim=1).cpu()

fig, axes = plt.subplots(4, 8, figsize=(16, 8))

mean = np.array(mean).reshape(1, 1, 3)
std = np.array(std).reshape(1, 1, 3)

for ax, img, label, pred in zip(axes.flatten(), imgs, label
s, pred_labels):
    img_np = img.permute(1, 2, 0).numpy()
    img_np = np.clip(img_np * std + mean, 0, 1)
    true_label = int(label.item())
    pred_label = int(pred.item())
    color = "green" if true_label == pred_label else "red"
    ax.imshow(img_np)
    ax.set_title(f"True: {class_dict[true_label]}\nPred: {c
lass_dict[pred_label]}", color=color)
    ax.axis("off")

plt.show()
```



f1-score 추가지표

```
from sklearn.metrics import f1_score, ConfusionMatrixDispla
y
```

```
all_preds = []
all_labels = []

model.eval()

with torch.no_grad():
    for X_batch, y_batch in test_dataloader:
        X_batch = X_batch.to(device)
        y_batch = y_batch.to(device)

        outputs = model(X_batch)
        preds = outputs.argmax(dim=1)

        all_preds.extend(preds.cpu().numpy())
        all_labels.extend(y_batch.cpu().numpy())

f1 = f1_score(all_labels, all_preds, average="macro")
```

```
print(f"F1-score: {f1:.4f}")
```

```
F1-score: 0.9315
```

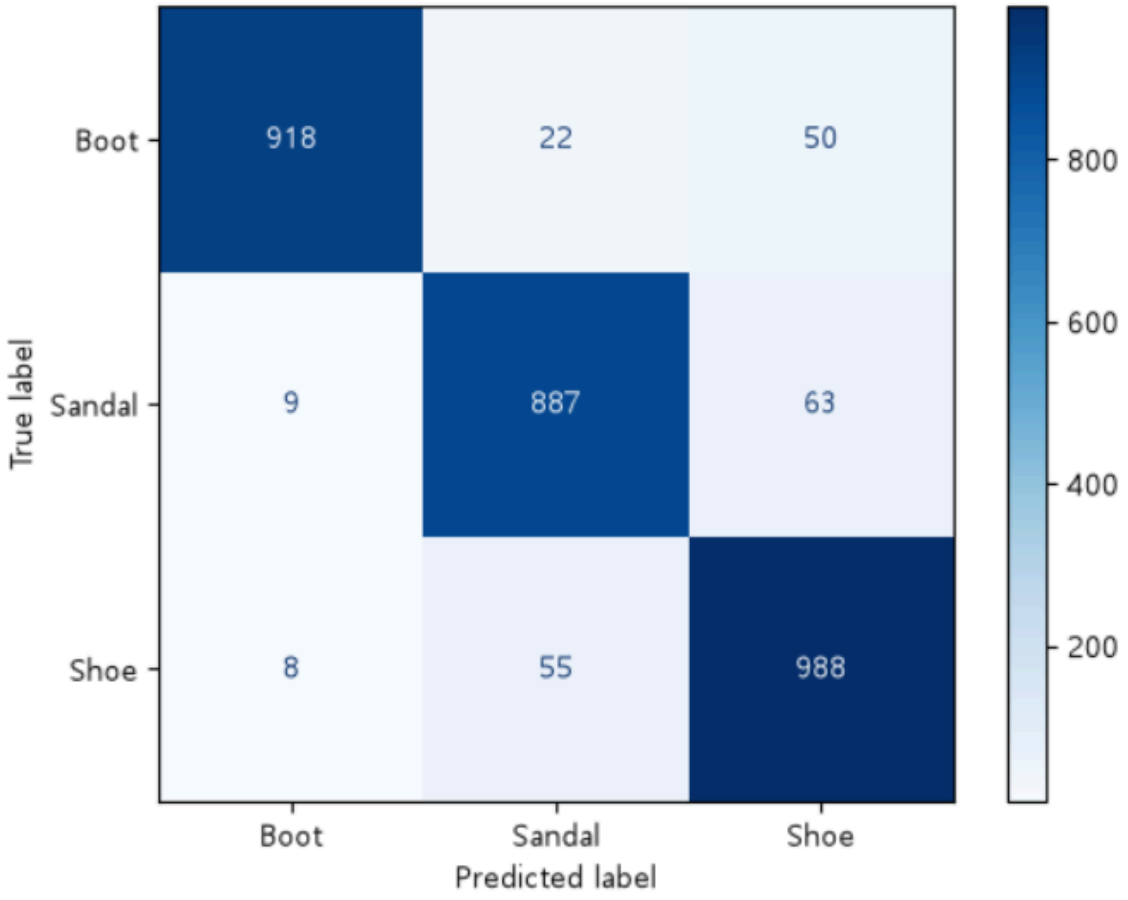
Macro F1-score가 0.9315이므로 세 가지 신발 클래스를 전반적으로 잘 분류하고 있다고 해석할 수 있다,

confusion matrix

```
cm = confusion_matrix(all_labels, all_preds)
cm
```

```
array([[918,  22,  50],
       [  9, 887,  63],
       [  8,  55, 988]])
```

```
cm_display = ConfusionMatrixDisplay(confusion_matrix=cm, display_labels=["Boot", "Sandal", "Shoe"])
cm_display.plot(cmap="Blues")
plt.show()
```



대각선 숫자: 정답을 맞힌 개수
대각선 밖 숫자: 다른 클래스로 틀리게 예측한 개수

대부분 잘 맞추는 것 같다. 그나마 샌달을 슈즈로 예측하는 오류가 가장 높게 나온것으로 보인다

오분류 찾기

```
model.eval()

incorrect_samples = []

with torch.no_grad():
    for imgs, labels in test_dataloader:
        imgs_device = imgs.to(device)

        preds = model(imgs_device)
        pred_labels = preds.argmax(dim=1).cpu()
        probs = torch.softmax(preds, dim=1).cpu()
```

```
        incorrect_mask = (pred_labels != labels)

        if incorrect_mask.any():
            for img, label, pred, prob in zip(imgs[incorrec
t_mask], labels[incorrect_mask], pred_labels[incorrect_mas
k], probs[incorrect_mask]):
                incorrect_samples.append({
                    "img": img,
                    "true_label": label.item(),
                    "pred_label": pred.item(),
                    "pred_prob": prob[pred.item()].item()
                })

# print(incorrect_samples)
```

```
print(incorrect_mask)
```

```
tensor([False, False, False,  True, False, False, False, False, False, False,
        False, False, False, False, False, False, False, False,  True, False,
        False, False, False, False])
```

오분류 얼마나 있는지 확인

```
for sample in incorrect_samples[:10]:
    print(f"정답 : {base_dataset.classes[sample["true_labe
l"]]} | 예측 : {base_dataset.classes[sample["pred_label"]]}
| 예측 확률 : {round(sample["pred_prob"], 2)}")
```

```
정답 : Boot | 예측 : Shoe | 예측 확률 : 0.77
정답 : Boot | 예측 : Sandal | 예측 확률 : 0.81
정답 : Sandal | 예측 : Shoe | 예측 확률 : 0.92
정답 : Sandal | 예측 : Shoe | 예측 확률 : 0.85
정답 : Sandal | 예측 : Shoe | 예측 확률 : 0.66
정답 : Sandal | 예측 : Shoe | 예측 확률 : 0.63
정답 : Shoe | 예측 : Sandal | 예측 확률 : 0.67
정답 : Shoe | 예측 : Sandal | 예측 확률 : 0.6
정답 : Sandal | 예측 : Shoe | 예측 확률 : 0.54
정답 : Boot | 예측 : Shoe | 예측 확률 : 0.46
```

Sandal → Shoe 오분류가 가장 많음 샌들을 신발로 잘못 예측하는 사례가 반복된다

모델 개선 아이디어

1. 입력 이미지 크기 조정

- 입력 이미지 크기를 224×224에서 256×256으로 변경해 성능을 비교.
- 근거: 이미지 해상도가 높아지면 신발의 작은 형태적 특징을 더 잘 보존해 분류 성능이 향상될 수 있다. 다만 연산량과 학습 시간이 증가할 수 있다.

2. Dropout 비율 조정

- 분류기의 Dropout 비율을 현재 0.2, 0.4에서 0.3 정도로 조정해 성능을 비교.
- 근거: 학습 중 일부 뉴런을 무작위로 비활성화해 특정 특징에 지나치게 의존하는 과적합을 줄일 수 있다.